



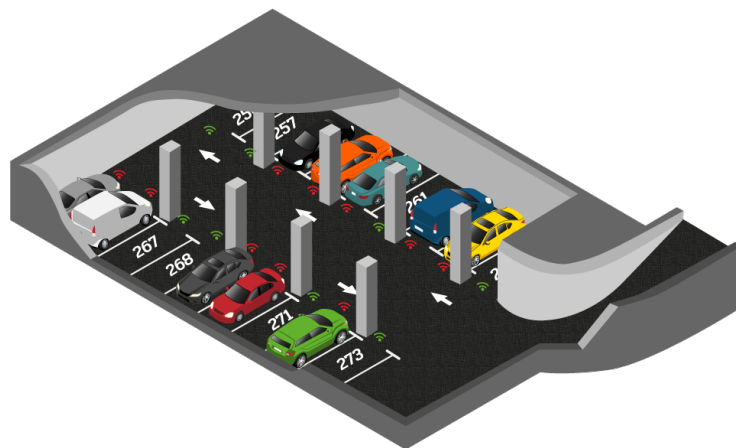
OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Projektbericht im Modul

Advanced IoT and Cyber Physical Systems

Smart Parking

Konzeption und Umsetzung eines intelligenten Parkhauses



Quelle: iot-shop.de – Smart Parking

Studiengang: Master Informatik
Verfasser: Oliver Masur
Dozent/in: Prof. Dr. Jube
Abgabe: 18.07.2026

Inhaltsverzeichnis

1	Projektbeschreibung	1
1.1	Zentrale Use Cases	1
1.2	Komponentenübersicht	1
2	Backend für Zahlung und Ausfahrt	2
2.1	Datenmodell	2
2.1.1	parking_sessions als Zustandsobjekt	2
2.1.2	RFID-Administratoren und Freigaben	2
2.2	Zahlungslogik in Node-RED	3
2.3	Ausfahrts-Flow	4
2.4	RFID-Freigabe im Backend	5
2.5	RFID-Hardware und Firmware	6
2.6	REST-Schnittstellen	7
2.6.1	Zahlung und Antwortmodell	7
2.6.2	RFID-Administration und Sessionzugriff	7
3	Fazit, Grenzen und Ausblick	8
3.1	Fazit	8
3.2	Grenzen und Security	8
3.3	Ausblick	8

1 Projektbeschreibung

Im Projekt entstand ein intelligentes Parkhausmodell mit Kennzeichenerkennung und zentraler Steuerung. Das System verwaltet die Einfahrt, die Stellplätze und die Ausfahrt. Hinzu kommen eine bewegungsabhängige Lichtsteuerung, die Überwachung der Stellplätze und mehrere Dashboards für den aktuellen Zustand des Parkhauses.

1.1 Zentrale Use Cases

- Kennzeichen erkennen, Kapazität prüfen und die Einfahrt freigeben,
- freie und belegte Stellplätze erfassen und anzeigen,
- Parkvorgang suchen, bezahlen und die Ausfahrt prüfen,
- die Schranke bei Bedarf durch einen RFID-Administrator freigeben,
- das Licht auf jeder Etage abhängig von erkannter Bewegung steuern,
- wichtige Zustandsänderungen zentral protokollieren.

1.2 Komponentenübersicht

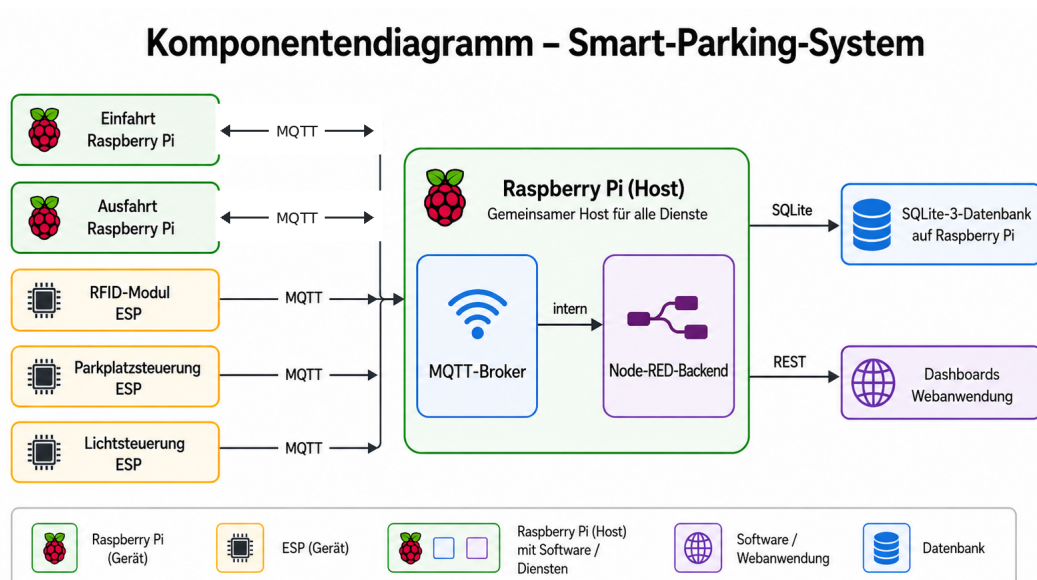


Abbildung 1: Komponentendiagramm des Smart-Parking-Systems
Quelle: gemeinsame Darstellung der Projektgruppe

Das Modell gliedert sich in die Bereiche Einfahrt, Ausfahrt, Stellplatzüberwachung und Lichtsteuerung. Die einzelnen Komponenten senden ihre Nachrichten über MQTT an das Node-RED-Backend. Node-RED verarbeitet die Abläufe und speichert die zugehörigen Daten in SQLite. Die Dashboards rufen die benötigten Informationen über REST-Schnittstellen ab.

2 Backend für Zahlung und Ausfahrt

Das Backend verbindet die Abläufe von der Einfahrt bis zur Ausfahrt. Dieser Teil wurde gemeinsam mit Jakob Frohwein umgesetzt. Die vorliegende Dokumentation betrachtet vor allem die Verarbeitung von Zahlungen, Ausfahrten und RFID-Freigaben. Während seines gesamten Aufenthalts wird ein Fahrzeug über eine Parksession geführt. So entstehen keine voneinander getrennten Datensätze für denselben Parkvorgang. Node-RED verarbeitet eingehende MQTT- und HTTP-Nachrichten, greift auf die Datenbank zu und antwortet den angeschlossenen Komponenten in einem einheitlichen Format. Die einzelnen Nodes reagieren auf Nachrichten oder andere Ereignisse und werden zu übersichtlichen Flows verbunden [1].

2.1 Datenmodell

SQLite benötigt keinen eigenen Serverprozess und speichert die Datenbank in einer einzelnen Datei [2]. Damit eignet sich die Datenbank gut für den Raspberry Pi, auf dem alle Backend-Flows laufen. Jeder Flow greift dort auf denselben gespeicherten Zustand zu.

Datenobjekt	Rolle im Prozess
<code>parking_sessions</code>	Parkvorgang mit Zahlungsstand, Freigabefenster und Abschluss
<code>user_admins</code>	zugelassene RFID-UIDs mit ihrem Aktivstatus
<code>rfid_admin_releases</code>	protokollierte Freigaben mit Admin und optionaler Session
<code>event_logs</code>	aufgezeichnete Ausfahrts-, Zahlungs- und RFID-Ereignisse

Tabelle 1: Gespeicherte Datenobjekte des Ausfahrtsprozesses

2.1.1 `parking_sessions` als Zustandsobjekt

Bereits bei der Einfahrt legt das Backend eine Session an. Sie bleibt offen, solange `exit_time` IS NULL gilt. Über das Kennzeichen finden die späteren Abläufe genau diesen Datensatz wieder. In `amount_due` steht der fällige Gesamtbetrag. `amount_paid` enthält die Summe der bisherigen Zahlungen. Nach der vollständigen Zahlung setzt das Backend `payment_time` und berechnet `exit_valid_until` mit einer Gültigkeit von 15 Minuten. Erst wenn die Ausfahrt erlaubt wurde, erhält `exit_time` einen Wert und die Session ist abgeschlossen. Bezahlung und Ausfahrt bleiben damit zwei getrennte Zustände.

2.1.2 RFID-Administratoren und Freigaben

Die Tabelle `user_admins` enthält den Namen, die `uid` und den Wert `active`. Nur ein aktiver Eintrag darf die Schranke freigeben. Für jede erfolgreiche Freigabe entsteht außerdem ein Datensatz in `rfid_admin_releases`. Er enthält `admin_id`, `rfid_uid` und `release_time`. Wenn ein passender Sessionkontext vorliegt, verweist `parking_session_id` auf diese Session. Bei einer direkten Freigabe ohne offene Session bleibt das Feld NULL. Die Feldnamen `uid` und `rfid_uid` unterscheiden sich, weil dies den dokumentierten Schnittstellen entspricht. Zusätzlich zeichnet das Backend Ereignisse wie `EXIT_BARRIER_DENIED`, `RFID_ADMIN_RELEASE` und `UNKNOWN_RFID` auf.

2.2 Zahlungslogik in Node-RED

Das Zahlungs-Dashboard sucht die offene Session über das vereinheitlichte Kennzeichen. Der Flow liest den fälligen Betrag und die bisherige Zahlung aus der Datenbank. Den neuen Wert aus `amount_paid_now` addiert er zu `amount_paid`. Frühere Zahlungen bleiben erhalten, sodass auch mehrere Teilzahlungen möglich sind. Der verbleibende Betrag wird mit $\max(0, \text{amount_due} - \text{amount_paid})$ berechnet.

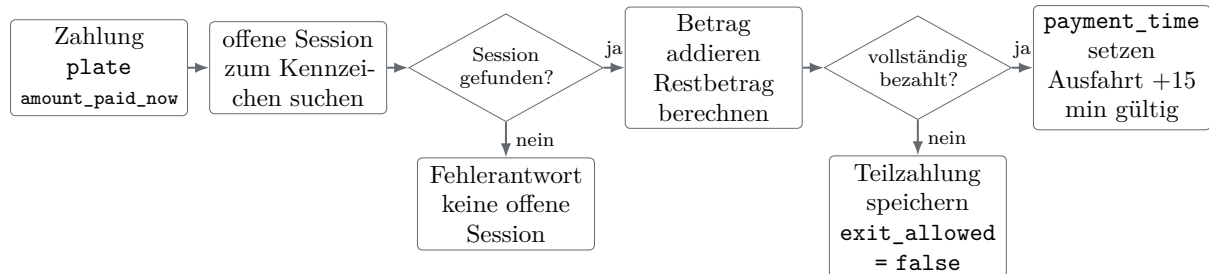


Abbildung 2: Zustandsübergänge beim Verbuchen einer Zahlung
Quelle: eigene Darstellung auf Grundlage der Projektarchitektur

Nach einer Teilzahlung speichert das Backend den neuen Gesamtstand. Sobald `amount_paid` mindestens `amount_due` erreicht, hält `payment_time` den Abschluss der Zahlung fest. Gleichzeitig begrenzt `exit_valid_until` die Ausfahrt auf 15 Minuten. Eine bezahlte Session bleibt deshalb nicht unbegrenzt zur Ausfahrt berechtigt. Bei einem späteren Aufruf kann der Flow den aktuellen Stand erneut aus der Datenbank lesen. Ein flüchtiger Node-RED-Kontext ist dafür nicht nötig.

Listing 1: Zahlungsanfrage am Kassenendpunkt

```
{
  "plate": "RAB1234",
  "amount_paid_now": 2.0
}
```

Listing 2: Antwort nach vollständiger Zahlung

```
{
  "success": true,
  "amount_paid_total": 3.0,
  "remaining_amount": 0.0,
  "exit_allowed": true
}
```

Die Antwort nennt neben dem Erfolg auch den Gesamt- und den Restbetrag. Damit kann das Dashboard den aktuellen Zahlungsstand anzeigen, ohne die Berechnung selbst auszuführen. Bei einer Überzahlung bleibt der Restbetrag bei null. Für einen späteren produktiven Betrieb müsste noch entschieden werden, ob das System eine Überzahlung ablehnt oder als Guthaben behandelt.

2.3 Ausfahrts-Flow

Die Kamera an der Ausfahrt sendet das erkannte Kennzeichen über `ausfahren/plate`. MQTT arbeitet nach dem Publish/Subscribe-Modell. Das Topic ordnet eine Nachricht den interessierten Empfängern zu [3]. Nach dem Einlesen der JSON-Daten bringt der Flow das Kennzeichen in dasselbe Format wie bei der Einfahrt. Danach sucht er eine offene Session mit `exit_time` IS NULL.

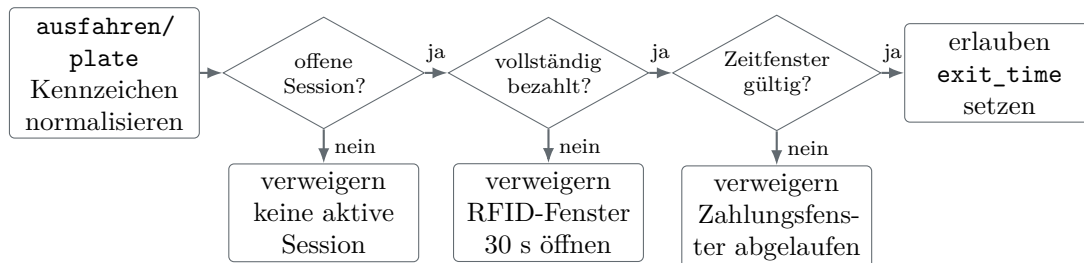


Abbildung 3: Prüfreihefolge vor dem Öffnen der Ausfahrt
Quelle: eigene Darstellung auf Grundlage der Projektarchitektur

Findet der Kennzeichen-Flow keine aktive Session, lehnt er die Ausfahrt zunächst ab. Ohne eine Session lässt sich kein Parkvorgang eindeutig zuordnen. Ein berechtigter RFID-Administrator kann die Schranke trotzdem direkt öffnen. Auch bei einer unbezahlten Session verweigert der Flow zuerst die Ausfahrt. Gleichzeitig startet ein 30 Sekunden langes RFID-Fenster. Innerhalb dieser Zeit kann ein Administrator genau diese Session freigeben. Nach einer vollständigen Zahlung vergleicht der Flow den aktuellen Zeitpunkt mit `exit_valid_until`. Ist das 15-Minuten-Fenster bereits abgelaufen, bleibt nur noch die Freigabe durch einen RFID-Administrator.

Das Backend setzt `exit_time` nur bei einer erlaubten Ausfahrt mit vorhandener Session. Dies geschieht entweder nach einer gültigen Zahlung oder nach einer sessionbezogenen RFID-Freigabe. Öffnet ein Administrator die Schranke ohne Session, bleibt jeder vorhandene Parkvorgang unverändert. Das Backend protokolliert die Entscheidungen als `EXIT_PLATE_DETECTED`, `EXIT_BARRIER_OPENED` oder `EXIT_BARRIER_DENIED`. Die Hardware erhält in jedem Fall dasselbe Antwortformat. Die beiden folgenden Listings zeigen die möglichen Antworten.

Listing 3: Freigabe an der Schranke

```
{
  "status": "allowed",
  "display_text": "Gute Fahrt"
}
```

Listing 4: Ablehnung an der Schranke

```
{
  "status": "denied",
  "display_text":
    "RAB1234 nicht erlaubt - RFID-Chip halten  
oder bezahlen (30s)"
}
```

Beide Antworten werden über `ausfahren/status` veröffentlicht. Der Raspberry Pi an der Ausfahrt steuert anschließend den Motor, die LEDs und das Display. Die Entscheidung über die Freigabe trifft weiterhin das Backend.

2.4 RFID-Freigabe im Backend

Mit RFID kann ein Administrator die Schranke unabhängig vom regulären Zahlungsweg freigeben. Der ESP8266 veröffentlicht die gelesene UID als einfachen Payload auf `ausfahren/rfid`. Node-RED vereinheitlicht das Format und sucht den passenden Datensatz in `user_admins`. Eine gelesene UID reicht für die Freigabe noch nicht aus. Der Datensatz muss vorhanden sein und `active = 1` enthalten.

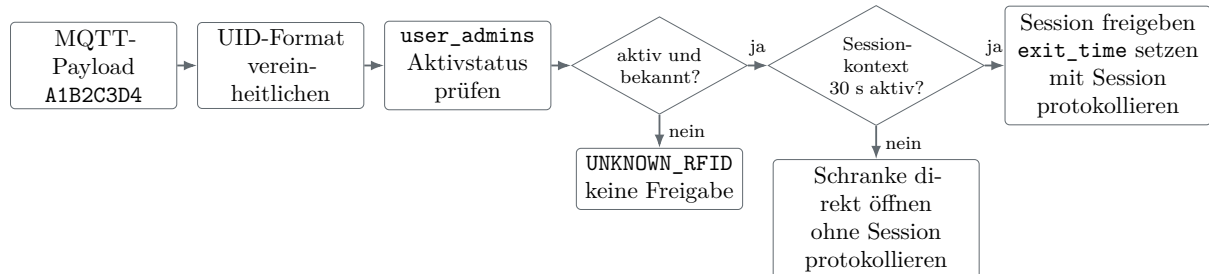


Abbildung 4: Berechtigungsprüfung einer manuellen RFID-Freigabe
Quelle: eigene Darstellung auf Grundlage der Projektarchitektur

Nach einer verweigerten Ausfahrt merkt sich der Flow die betroffene Session für 30 Sekunden. Scant ein aktiver Administrator in diesem Zeitraum seinen RFID-Chip, ordnet das Backend die Freigabe genau dieser Session zu. Es setzt `exit_time` und öffnet die Schranke. Ohne offene Session oder nach Ablauf der 30 Sekunden öffnet der Administrator die Schranke direkt. Ein Parkvorgang wird in diesem Fall nicht verändert. Der gespeicherte Kontext bestimmt also nur die Zuordnung zu einer Session. Die Berechtigung des Administrators hängt weiterhin von seinem aktiven Eintrag ab.

Node-RED meldet beide Arten der Freigabe über `ausfahren/status`. Admin-ID, UID und Zeitpunkt werden in `rfid_admin_releases` gespeichert. Bei einer sessionbezogenen Freigabe enthält der Datensatz zusätzlich die Session-ID. Bei einer direkten Freigabe steht in `parking_session_id` der Wert NULL. In beiden Fällen zeichnet das Backend das Ereignis `RFID_ADMIN_RELEASE` auf. Ein abgelaufener Kontext kann deshalb keine frühere Session schließen.

Prüfung	Antwort	Speicherung
aktiver Admin, Sessionkontext gültig	<code>allowed</code> , sessionbezogene RFID-Freigabe	Datensatz mit Session-ID und <code>RFID_ADMIN_RELEASE</code>
aktiver Admin, kein gültiger Sessionkontext	<code>allowed</code> , direkte RFID-Freigabe	Datensatz mit <code>parking_session_id</code> = NULL und Ereignis
UID unbekannt oder deaktiviert	<code>denied</code> , RFID unbekannt	keine Freigabe und protokollierte Ablehnung

Tabelle 2: Ergebnisse der RFID-Berechtigungsprüfung

Die Entscheidung über eine Freigabe liegt vollständig im Backend. Dort wirken Aktivierungen und Sperrungen sofort für alle Lesegeräte. Eine Änderung der Firmware ist nicht nötig. Die Ausfahrtseinheit liest nur `status` und `display_text` aus der Antwort. Danach steuert sie den Motor, die LEDs und das Display.

2.5 RFID-Hardware und Firmware

Der RFID-Aufbau besteht aus einem RC522-Lesemodul, einer Karte, einem Schlüsseltransponder und einem ESP8266 mit WLAN. Karte und Transponder arbeiten mit 13,56 MHz. Die Bauteile sind über ein Breadboard verbunden. Das RC522-Modul wird mit 3,3 V versorgt und nutzt die SPI-Schnittstelle des Mikrocontrollers [4].



Verkabelung über das Breadboard

RC522	ESP8266	Funktion
3.3V	3.3V	Spannungsversorgung
GND	GND	Masse
RST	D0	Reset
SDA/SS	D8	SPI-Chip-Select
SCK	D5	SPI-Takt
MISO	D6	Daten zum ESP8266
MOSI	D7	Daten zum RC522

Abbildung 5: RC522-RFID-Kit und Breadboard-Verkabelung mit dem ESP8266
Bildquelle: OpenELAB [4]

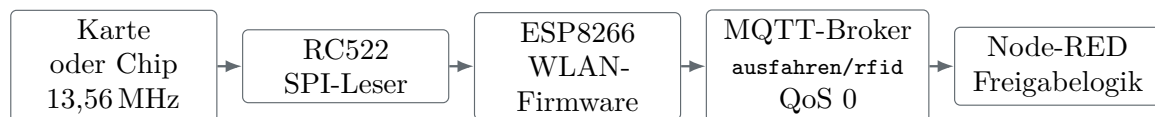


Abbildung 6: Datenweg vom Transponder bis zur Freigabelogik
Quelle: eigene Darstellung auf Grundlage der Implementierung

Nach dem Einschalten initialisiert die Firmware SPI und den MFRC522. Anschließend verbindet sie sich mit dem WLAN und dem MQTT-Broker. Für die Verbindung gilt ein Keepalive von 30 Sekunden. Die Firmware setzt die einzelnen UID-Bytes als zweistellige Hexadezimalwerte zusammen und wandelt die Buchstaben in Großbuchstaben um. Wiederholte Scans derselben UID werden für zwei Sekunden unterdrückt. Die MQTT-Nachricht wird mit QoS 0 und ohne Retain verschickt. Node-RED liest den Rohwert als `uid`.

Listing 5: UID-Aufbereitung und MQTT-Versand

```

String uid = readUidAsString();
unsigned long currentTime = millis();

bool duplicate =
    uid == lastUid &&
    currentTime - lastUidTime < duplicateBlockTime;

if (!duplicate && mqttClient.connected()) {
    mqttClient.publish(mqttTopic, uid, false, 0);
    lastUid = uid;
    lastUidTime = currentTime;
}
  
```

2.6 REST-Schnittstellen

Über die REST-API greifen das Admin- und das Zahlungs-Dashboard auf die Node-RED-Flows zu. REST und MQTT verwenden dieselben Datenbanktabellen. Deshalb arbeiten die Dashboards und die Schrankenlogik immer mit demselben Stand. Die Tabelle zeigt die Schnittstellen für Zahlung, Ausfahrt und RFID.

HTTP	Ressource	Verwendung
GET	/api/admin/rfid-admins	RFID-Administratoren laden
POST	/api/admin/rfid-admins	Namen, UID und Aktivstatus anlegen
PATCH	/api/admin/rfid-admins/:uid	Chip aktivieren oder deaktivieren
GET	/api/admin/rfid-releases	protokollierte Freigaben abrufen
GET	/api/payments/open-sessions	offene Parkvorgänge für die Zahlung laden
GET	/api/payments/session/by-plate/:plate	offene Session zum Kennzeichen suchen
POST	/api/payments/pay	Teil- oder Vollzahlung verbuchen
PATCH	/api/payments/session/:sessionId	Zahlungsstand einer Session bearbeiten
GET	/api/admin/parking-sessions/:sessionId	einzelne Session administrativ laden
PATCH	/api/admin/parking-sessions/:sessionId	freigegebene Sessionfelder ändern

Tabelle 3: HTTP-Zugriffe für Kassen- und Administrationsfunktionen

2.6.1 Zahlung und Antwortmodell

Der Endpunkt `POST /api/payments/pay` erwartet `plate` und `amount_paid_now`. Das Backend vereinheitlicht das Kennzeichen und prüft, ob der Betrag positiv ist. Danach sucht es genau eine offene Session. Die Antwort meldet den Erfolg und enthält außerdem die Session-ID, den Sollbetrag, die gesamte Zahlung, den Restbetrag und `exit_allowed`. Das Dashboard muss die 15-Minuten-Regel nicht selbst berechnen.

Listing 6: Rückgabe des Zahlungsendpunkts

```
{
  "success": true,
  "plate": "RAB1234",
  "session_id": 12,
  "amount_due": 3.0,
  "amount_paid_total": 3.0,
  "remaining_amount": 0.0,
  "exit_allowed": true
}
```

2.6.2 RFID-Administration und Sessionzugriff

Beim Anlegen eines Administrators erwartet die API die Felder `name`, `uid` und `active`. Der `PATCH`-Endpunkt findet den Datensatz über die UID. Mit `GET /api/admin/rfid-releases` lassen sich manuelle Eingriffe später nachvollziehen. Die `PATCH`-Endpunkte für Sessions dürfen nur ausdrücklich freigegebene Felder ändern. Eine Nutzereingabe darf nicht ungeprüft in die gesamte Datenbankzeile übernommen werden.

Bei Fehlern sollte die API zwischen ungültigem JSON, fehlenden Pflichtfeldern, nicht gefundenen Sessions, doppelten RFID-UIDs und unzulässigen Beträgen unterscheiden. Zu jedem Fehler gehören ein passender HTTP-Status und ein einheitlich aufgebautes JSON-Objekt. Die Projektunterlagen legen dieses Fehlerformat noch nicht verbindlich fest. Aus diesem Grund wird es hier nicht als bestehender Vertrag beschrieben.

3 Fazit, Grenzen und Ausblick

3.1 Fazit

Das Backend führt Zahlung, Ausfahrtsentscheidung und RFID-Freigabe in einem gemeinsamen Ablauf zusammen. Weil die Sessions in der Datenbank gespeichert sind, bleiben sie auch nach einem Neustart erhalten. Mehrere Teilzahlungen werden addiert. Nach der vollständigen Zahlung beginnt ein 15-Minuten-Fenster, und erst die erlaubte Ausfahrt schließt die Session. Zusätzlich können aktive RFID-Administratoren eine bestimmte Session freigeben oder die Schranke ohne offene Session öffnen. Das Backend prüft in beiden Fällen die Berechtigung und protokolliert den Vorgang.

Die eingesetzten Technologien übernehmen unterschiedliche Aufgaben. MQTT überträgt zeitnahe Ereignisse zwischen der Hardware und dem Backend. Über REST greifen die Dashboards auf Daten zu und können freigegebene Werte ändern. SQLite speichert den gemeinsamen Zustand. Die Ausfahrtshardware benötigt nur die einheitlichen Antworten mit `status` und `display_text`. Sie muss den internen Aufbau des Backends nicht kennen.

3.2 Grenzen und Security

Beim Prototyp stand zunächst die funktionierende Verbindung aller Komponenten im Vordergrund. MQTT, REST und die Netzwerkzugriffe sind noch nicht vollständig abgesichert. Für einen späteren Betrieb müssen daher mehrere Risiken berücksichtigt werden.

- Ohne Transportverschlüsselung können Kennzeichen, UID und Freigaben mitgelesen oder verändert werden.
- Eine statische RFID-UID kann kopiert oder erneut gesendet werden. Das 30-Sekunden-Fenster begrenzt nur die Zuordnung zu einer Session.
- Nicht authentifizierte Admin- und PATCH-Endpunkte könnten Zahlungs-, Aktiv- oder Freigabedaten unberechtigt verändern.
- Unbegrenzt wiederholbare Anfragen begünstigen Doppelzahlungen, automatisierte Zugriffsversuche und hohen Ressourcenverbrauch.

Die MQTT-Spezifikation überlässt es der jeweiligen Systemarchitektur, Authentifizierung, Zugriffsrechte und vertrauliche Übertragung umzusetzen. Für geschützte Verbindungen empfiehlt sie unter anderem Zugriffskontrollen pro Topic und TLS [3]. Bei der REST-API sind vor allem die Rechteprüfung für Objekte und Funktionen sowie eine sichere Konfiguration wichtig [5]. Eine produktive Version sollte TLS für MQTT und HTTPS verwenden. Der Broker braucht feste Zugriffsregeln, jedes Gerät eine eigene Identität und das Dashboard eine Anmeldung mit passenden Rollen. Geprüfte Eingaben und eine Begrenzung wiederholter Anfragen gehören ebenfalls dazu. Auch RFID-Freigaben benötigen einen Schutz gegen erneut gesendete Nachrichten. Eine Möglichkeit wären kurzlebige Prüfcodes, die nur einmal gültig sind und die statische UID ergänzen.

3.3 Ausblick

Der Prototyp deckt bereits wichtige Funktionen eines Smart-Parkhauses ab und bietet eine Grundlage für eine reale Umsetzung. Der nächste Schritt wäre die Erweiterung auf viele Stellplätze und einen zuverlässigen Dauerbetrieb. Dafür braucht es robuste Sensoren, wartbare Hardware und eine Anbindung an vorhandene Zahlungs- und Parkleitsysteme. So kann aus dem Modell ein alltagstaugliches System entstehen, das Abläufe automatisiert, freie Plätze sichtbar macht und das Parken für Betreiber und Parkende vereinfacht.

Literatur

- [1] OpenJS Foundation. *Node-RED Concepts*. URL: <https://nodered.org/docs/user-guide/concepts> (besucht am 17.07.2026).
- [2] SQLite Consortium. *About SQLite*. URL: <https://www.sqlite.org/about.html> (besucht am 17.07.2026).
- [3] Andrew Banks, Ed Briggs, Ken Borgendale und Rahul Gupta. *MQTT Version 5.0*. OASIS Open. März 2019. URL: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (besucht am 17.07.2026).
- [4] OpenELAB Technology Ltd. *RFID Kit RC522 with Reader, Chip and Card (13,56MHz)*. URL: <https://openelab.io/products/rfid-kit-rc522> (besucht am 17.07.2026).
- [5] OWASP Foundation. *OWASP API Security Top 10 – 2023*. 2023. URL: <https://owasp.org/API-Security/> (besucht am 17.07.2026).

Abbildungsverzeichnis

1	Komponentendiagramm des Smart-Parking-Systems	1
2	Zustandsübergänge beim Verbuchen einer Zahlung	3
3	Prüfreihefolge vor dem Öffnen der Ausfahrt	4
4	Berechtigungsprüfung einer manuellen RFID-Freigabe	5
5	Hardwareaufbau des RFID-Lesers	6
6	Datenweg des RFID-Scans	6

KI-Klausel

ChatGPT wurde als Hilfsmittel bei der sprachlichen Ausarbeitung, der grafischen Aufbereitung einzelner Abläufe und der LaTeX-Formatierung verwendet. Inhaltliche Konzeption und Implementierung des Projekts wurden eigenständig durchgeführt. Die fachliche Verantwortung liegt beim Verfasser, sämtliche Vorschläge wurden geprüft und bei Bedarf angepasst.